Astana IT University
Computer Vision

Hand Gesture Recognition with ConvNeXt and Grad CAM

Assignment 2 report
Prepared by Zhanibek Yskak and Ivan Kuznetsov
Instructor Nurym Kaiyrkhan
November 2026

Abstract

This project upgrades the Assignment 1 hand gesture recognition system from a lightweight MobileNetV2 baseline to ConvNeXt-Tiny. It recognizes three static gestures: like, palm and peace. To address limited training diversity, the original four-subject collection is supplemented with 1,050 HaGRID images. The resulting split contains 1,048 training images, 263 validation images and a separate 64-image test set from Subject 4, Damir.

On this held-out subject, ConvNeXt-Tiny achieves 98.44% accuracy and a macro-F1 score of 0.9853, compared with 93.75% and 0.9419 for MobileNetV2. The accuracy gain is 4.69 percentage points. Grad-CAM explains spatial evidence for each prediction, while a Streamlit application supports image upload, webcam snapshots and side-by-side model comparison. The reported CPU forward-pass latency is 38.45 ms for ConvNeXt-Tiny; interactive explanations require additional computation.

Introduction and background

Vision-based hand gesture recognition supports touchless human-computer interaction and can form part of robotic, assistive and immersive interfaces. The HaGRID study connects gesture recognition with device control and interactive services [1]. This assignment focuses on still-image classification of thumbs up, an open hand and a V-sign, rather than complete sign-language translation.

Assignment 1 used photographs from four individuals under relatively similar indoor conditions. Its error analysis raised concerns about background correlation, lighting variation and hand orientation. The upgrade therefore combines a stronger classifier, a larger training collection, visual explanations and an interactive demonstration. The term SOTA identifies the project’s ConvNeXt upgrade; the experiment compares two models on one project benchmark.

1 Dataset evolution and benchmark splits

The original collection contains 325 images from four subjects. The expanded corpus adds 350 HaGRID images for each of the three classes, giving 1,050 public images and 1,375 images overall. HaGRID was designed to include varied people, scenes and lighting conditions [1].

The training partition combines 208 original photographs from Subjects 1–3 with 840 public images. Validation combines another 53 original photographs with 210 public images. Subject 4, Damir, contributes all 64 test images and is held out from the original training and validation photographs.

Split	A1 images	HaGRID images	Total	Share	Role in pipeline
Training	208 Subjects 1–3	840 280/class	1,048	76.2%	Parameter optimization
Validation	53 Subjects 1–3	210 70/class	263	19.1%	Model selection and checkpointing
Test	64 Subject 4	0	64	4.7%	Unseen subject evaluation
Complete corpus	325	1,050	1,375	100.0%	All three partitions

Table 1. Dataset composition and the role of each split. Percentages are rounded to one decimal place.

Evaluation design

Keeping the test subject separate tests whether the learned representation transfers to an unseen person. The same test set is used for both models, so differences in accuracy are not caused by changing the evaluation images. The 64 test photographs contain 20 like, 18 palm and 26 peace examples.

The data preparation script uses a fixed split seed and reserves original files with the damir_ prefix for testing. The remaining original images and the public sample are divided between training and validation. This separation reduces identity overlap with the test subject; a complete leakage audit would additionally check file identities, near-duplicates and source provenance.

Data interpretation

The public sample is balanced across the three selected gestures. The test set is slightly imbalanced, which is why the analysis reports macro-averaged precision, recall and F1 as well as accuracy. Each class contributes equally to a macro average, regardless of its number of images.

The expanded data are intended to reduce the dependence on a small indoor collection. Diversity of image sources alone does not establish equal performance across demographic groups. Such a conclusion would require subgroup labels and independent subgroup evaluation.

2 Model architecture and experimental setup

MobileNetV2 provides the lightweight baseline. Its inverted residual blocks use depthwise convolution in an expanded feature space and linear bottlenecks between blocks. In this project, the classifier is adapted to produce three class scores.

ConvNeXt-Tiny is the upgraded convolutional backbone. ConvNeXt modernizes a residual convolutional architecture using design choices informed by vision Transformers [2]. The Vision Transformer represents images as sequences of embedded patches [3]. The project uses ConvNeXt-Tiny to combine these design ideas with a convolutional feature hierarchy.

ConvNeXt components

The 4 × 4 convolutional stem uses stride 4, reducing a 224 × 224 image to a 56 × 56 feature grid. Blocks combine 7 × 7 depthwise convolutions with a channel expansion factor of four, LayerNorm and GELU activation. Large convolutional kernels expand the local receptive field while retaining a convolutional feature hierarchy. These design choices do not by themselves guarantee robustness to hand pose or background variation.

Training procedure

Both models are adapted from ImageNet-pretrained weights and trained with AdamW, which separates weight decay from the adaptive gradient update. The learning rate is 0.0001 and weight decay is 0.01. A CosineAnnealingLR scheduler reduces the learning rate over training. Validation performance is used to select the checkpoint, and the two selected models are evaluated on the same 64 test images.

Training inputs are resized and cropped to 224 × 224 pixels with horizontal flips, rotation and color variation. Evaluation uses a deterministic resize and center crop followed by ImageNet normalization. This keeps the inference preprocessing consistent for the two classifiers.

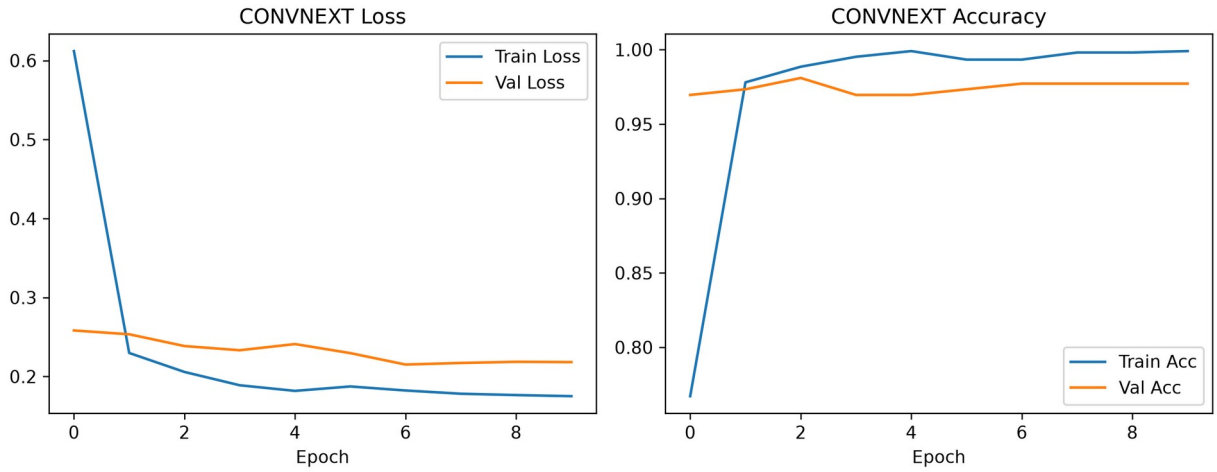


Figure 1. ConvNeXt-Tiny training and validation loss and accuracy over ten plotted epochs, indexed 0–9.

The recorded peak validation accuracy is 98.10%. Training loss falls rapidly, while validation loss remains higher and later stabilizes. The curves show successful adaptation on this split, but they are not sufficient to rule out overfitting to its subjects or capture conditions.

3 Comparative benchmark results

ConvNeXt-Tiny correctly classifies 63 of the 64 test images, compared with 60 for MobileNetV2. Accuracy increases by 4.69 percentage points and macro-F1 by 0.0434. These are results for the held-out subject and should not be interpreted as a population-wide accuracy estimate.

Metric	MobileNetV2	ConvNeXt-Tiny
Test accuracy	93.75% (60/64)	98.44% (63/64)
Macro precision	0.9444	0.9841
Macro recall	0.9487	0.9872
Macro-F1	0.9419	0.9853
CPU forward latency	26.81 ms	38.45 ms
Approximate forward throughput	37.3 FPS	26.0 FPS

Table 2. Aggregate classification scores and reported CPU forward-pass performance.

Gesture	Base precision	Base recall	Base F1	SOTA precision	SOTA recall	SOTA F1
like Thumbs up	0.8333	1.0000	0.9091	0.9524	1.0000	0.9756 (+0.0665)
palm Open hand	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000 (Parity)
peace V-sign	1.0000	0.8462	0.9167	1.0000	0.9615	0.9804 (+0.0637)

Table 3. Per-class precision, recall and F1. Base denotes MobileNetV2 and SOTA denotes ConvNeXt-Tiny.

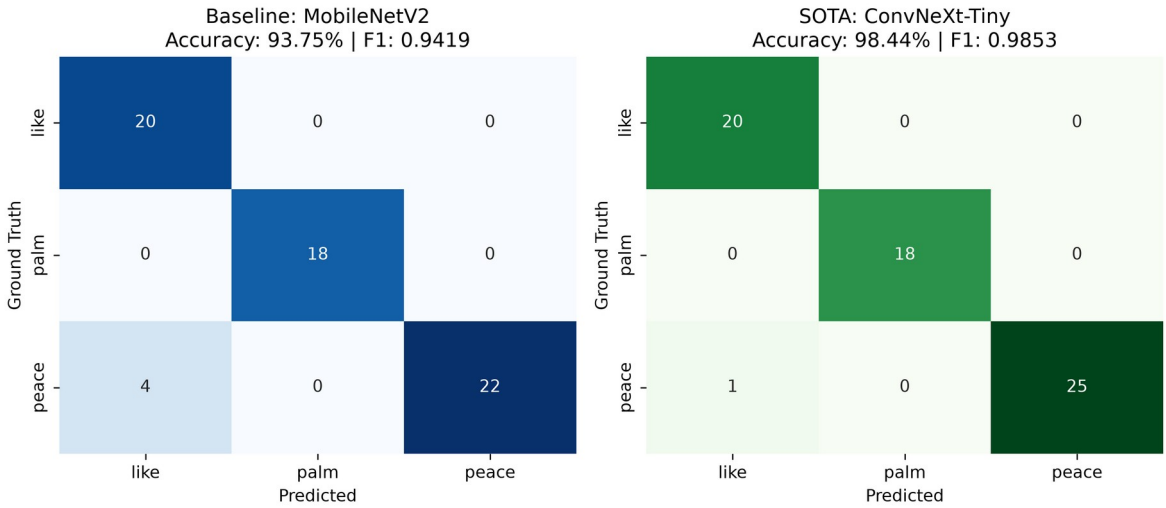


Figure 2. Confusion matrices on Subject 4. Rows represent true classes and columns represent predictions.

Both models correctly identify all like and palm images. The number of peace images predicted as like falls from four to one. ConvNeXt-Tiny therefore reduces this error but does not eliminate it. Its higher accuracy comes with an additional 11.64 ms of reported CPU forward latency.

4 Visual explainability with Grad CAM

Grad-CAM uses class-score gradients at a convolutional feature layer to form a coarse localization map [5]. The implementation averages these gradients across spatial positions, weights the feature maps by the resulting channel importance values and applies ReLU. The final map is resized and blended with the input image. Later work on LayerCAM combines information from different layers to improve localization [4]; this project uses Grad-CAM.

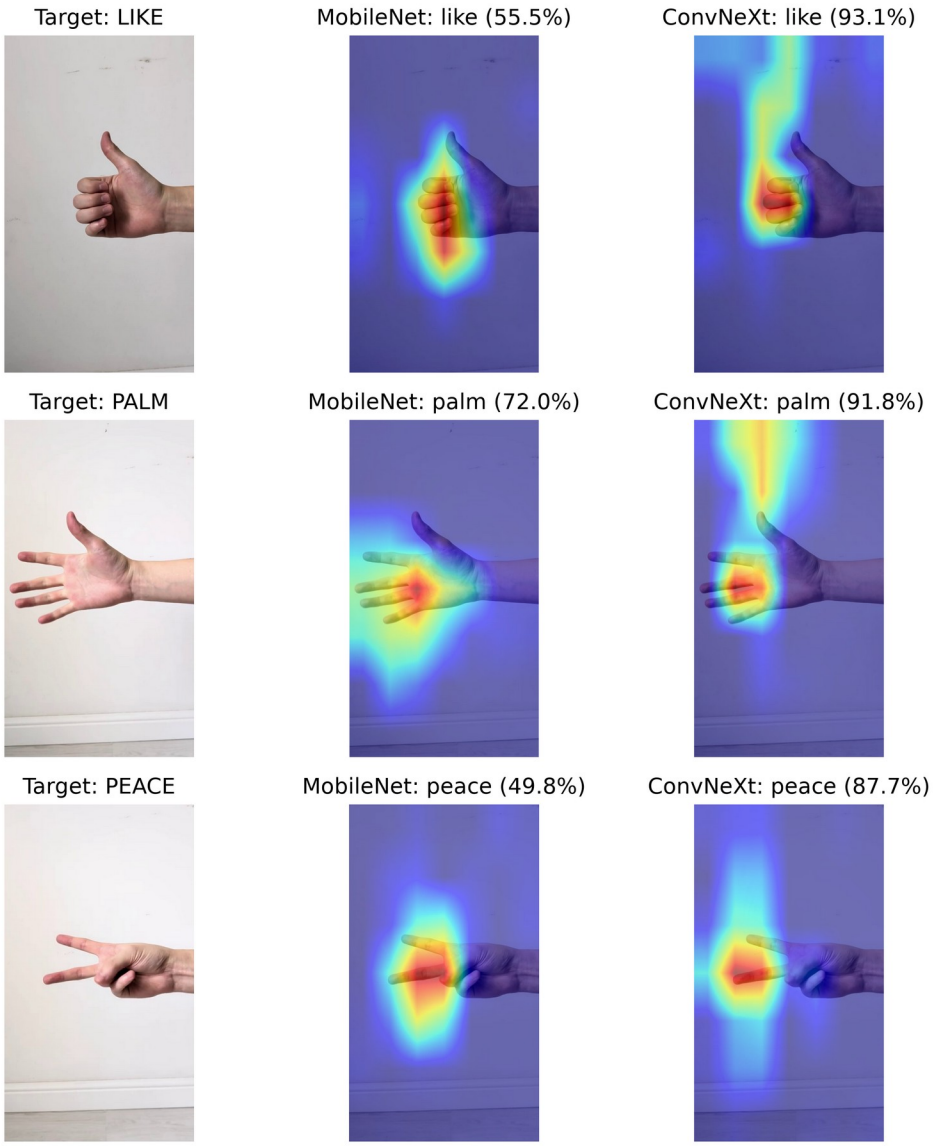


Figure 3. Original images and Grad-CAM overlays for like, palm and peace. Columns compare the original image, MobileNetV2 and ConvNeXt-Tiny.

The examples show hand-related regions contributing to both classifiers, including the thumb and extended fingers. ConvNeXt-Tiny assigns higher class scores in these examples: 93.1%, 91.8% and 87.7%, compared with 55.5%, 72.0% and 49.8% for MobileNetV2. Some highlighted regions also extend beyond the hand. Grad-CAM provides qualitative evidence, rather than a guarantee that background cues have been removed [5].

5 Interactive application and responsible use

The Streamlit application in app/app.py connects trained checkpoints to an accessible interface. Users can upload a JPG or PNG image or capture a still image through the webcam control. The sidebar selects MobileNetV2, ConvNeXt-Tiny or a side-by-side comparison and displays the compute device and checkpoint status.

Prediction and explanation workflow

For a single model, the interface presents the predicted gesture, class score, timing, original image, Grad-CAM overlay and class probabilities. The comparison mode processes the same image with both models and places their predictions and heatmaps beside each other. This makes the architectural trade-off visible without changing the input.

The opacity slider adjusts the blend between the photograph and the heatmap. It changes the visualization rather than the learned model. The webcam input captures snapshots; it does not provide continuous video-stream classification.

Latency interpretation

The forward-only CPU measurements are 26.81 ms for MobileNetV2 and 38.45 ms for ConvNeXt-Tiny, equivalent to approximately 37.3 and 26.0 forward passes per second. The application timer also includes the backward pass needed for Grad-CAM. Loading, preprocessing and interface rendering affect the complete interaction, so the forward-only figures should not be presented as guaranteed application frame rates.

Ethics and bias

Combining the original photographs with HaGRID adds variation in subjects and scenes, but no skin-tone or other demographic subgroup scores are available. Balanced recognition across Fitzpatrick phototypes therefore remains an evaluation objective. Future testing should include multiple independent people under different lighting and background conditions.

The application code processes uploaded images and webcam captures in memory rather than intentionally saving them as image files. Images still pass from the browser to the Streamlit server. A local deployment keeps that processing on the local machine; a remote deployment needs appropriate access and transport protection. Camera use should be voluntary and limited to the images needed for recognition.

Limitations

Motion blur, occlusion and severe out-of-plane rotation can make gesture shape difficult to distinguish. The three-class model always selects a known gesture, even when the input is unrelated. Confidence scores are not calibrated guarantees of correctness. The heatmap is also stretched over the original image after center-crop preprocessing, so non-square inputs can show imperfect spatial alignment.

6 Reproducibility and conclusion

Project resources

The repository contains the application, data preparation, model training, evaluation and Grad-CAM implementation. Model weights are distributed separately because the checkpoint files are excluded from Git. Place baseline_mobilenet.pth and sota_convnext.pth in the weights directory before launching the application.

Repository: https://github.com/hulahup4ikk/hand-gesture-recognition-sota

Local setup and demonstration

```
git clone https://github.com/hulahup4ikk/hand-gesture-recognition-sota.git
cd hand-gesture-recognition-sota
pip install -r requirements.txt
streamlit run app/app.py
```

Open http://localhost:8501, choose a model, upload a gesture image and inspect the prediction and Grad-CAM overlay. Switch to the comparison view to evaluate both models on the same input. The demo video follows this workflow and explains the recorded benchmark.

Reproducing dataset preparation and evaluation

Restore the original photographs with their subject identifiers before rebuilding the dataset. The public sample alone does not restore the 64-image held-out test set. Run the preparation script, then evaluate each checkpoint on the restored test partition.

```
python data/prepare_dataset.py
python src/evaluate.py --model convnext \
  --weights weights/sota_convnext.pth
python src/evaluate.py --model mobilenet \
  --weights weights/baseline_mobilenet.pth
```

For a repeatable training run, retain the exact split file lists, software versions, training command and random seeds alongside the checkpoint. The results and figures in this report describe the recorded experiment; a rerun depends on recovering the same data and configuration.

Conclusion

The upgrade combines a larger dataset, ConvNeXt-Tiny, visual explanations and a usable web interface. On the 64-image held-out-subject benchmark, ConvNeXt-Tiny reaches 98.44% accuracy and 0.9853 macro-F1, improving the baseline by 4.69 percentage points and reducing four classification errors to one. The main gain is fewer peace gestures being confused with like.

MobileNetV2 retains an advantage in reported forward latency. ConvNeXt-Tiny is the stronger classifier in this experiment, while deployment choice also depends on hardware and the cost of generating explanations. The next evaluation should expand the number of test subjects, measure performance across capture conditions and add an unknown-gesture condition.

References

1. Kapitanov, A., Kvanchiani, K., Nagaev, A., Kraynov, R., and Makhliarchuk, A. (2024). HaGRID -- HAnd Gesture Recognition Image Dataset. WACV 2024, 4572–4581. https://arxiv.org/abs/2206.08219

2. Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., and Xie, S. (2022). A ConvNet for the 2020s. CVPR 2022, 11976–11986. https://arxiv.org/abs/2201.03545

3. Dosovitskiy, A., et al. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR 2021. https://arxiv.org/abs/2010.11929

4. Jiang, P.-T., Zhang, C.-B., Hou, Q., Cheng, M.-M., and Wei, Y. (2021). LayerCAM: Exploring Hierarchical Class Activation Maps for Localization. IEEE Transactions on Image Processing, 30, 5875–5888. https://doi.org/10.1109/TIP.2021.3089943

5. Taimeskhanov, M., Sicre, R., and Garreau, D. (2025). CAM-Based Methods Can See through Walls (Extended Abstract). IJCAI 2025, 10937–10941. https://doi.org/10.24963/ijcai.2025/1220

Implementation source

hulahup4ikk. Hand Gesture Recognition SOTA and Explainability. Source code, project figures and recorded metrics. Repository snapshot 95fb48f136327de80618ee7d759bf90ecf95b7d5.

https://github.com/hulahup4ikk/hand-gesture-recognition-sota/tree/95fb48f136327de80618ee7d759bf90ecf95b7d5